Decision tables

Overview

Decision tables drive most of the data-driven behaviour in SWAT+: scheduled and conditional management on land use objects, reservoir release rules, scenario land use change, and flow control. A decision table lists conditions, alternative rule sets across those conditions, and the actions to fire when an alternative is matched. The runtime walks every condition, narrows the active alternatives, and fires each action whose action-outcome entry under any surviving alternative is `y`.

There are four decision table classes, each with its own file and its own array of in-memory tables:

Class	Input file	Array	Use
Land use management	`lum.dtl`	`dtbl_lum`	Plant, fert, till, harvest, kill operations on HRUs
Reservoir release	`res_rel.dtl`	`dtbl_res`	Daily reservoir release rules
Scenario land use	`scen_lu.dtl`	`dtbl_scen`	Land use change scenarios
Flow control	`flo_con.dtl`	`dtbl_flo`	Channel and routing flow rules

All four share the same derived type `decision_table` defined in `conditional_module.f90`.

Process equations

Data model

A decision table holds:

- `conds`, `alts`, `acts`: number of conditions, alternatives, and actions.
- `cond(conds)`: each condition has a variable name (`var`), an object reference (`ob`, `ob_num`), a comparison limit (`lim_const`, `lim_var`, `lim_op`).
- `alt(conds, alts)`: a string per condition per alternative giving the comparison operator (`<`, `>`, `<=`, `>=`, `=`, `/=`) or `-` for "any".
- `act(acts)`: each action has a type name (`typ`), an object reference, a name, an option, a constant `const`, an application fraction `fp`, and a file pointer.
- `act_outcomes(acts, alts)`: a `y` or `n` per action per alternative giving whether to fire the action if that alternative survives.
- `act_typ(acts)`, `act_app(acts)`: integer pointers resolved at read time (for instance, fertiliser type from the fert database, harvest operation type).
- `act_hit(alts)`: scratch array set during evaluation. Starts as `y` for every alternative; conditions flip it to `n` when not satisfied.

Condition evaluation

The driver `conditions.f90` walks `d_tbl%cond(1:conds)` and uses a `select case` on the condition variable name. Supported variables include:

```
w_stress n_stress p_stress phu_plant phu_base0 precip_cur precip_next
plant_gro plant_name_gro days_plant days_harv days_irr days_act
day_start irr_year slope soil_water tile_flo aqu_dep biom_lai
month jday year_seq year_rot year_cal year_dur prob prob_unif
prob_norm land_use weed_pres ...
```

(The full set is the case list in `conditions.f90`.)

For each variable the routine fetches the current value (from plant, soil, climate, time, or scenario state) and calls one of the comparison helpers:

- `cond_real.f90` for real-valued comparisons.
- `cond_real_c.f90` for real-valued comparisons against a constant in a paired condition.
- `cond_integer.f90`, `cond_integer_c.f90` for integer-valued comparisons.

`cond_real.f90` implements the alternative narrowing:

```
do ialt = 1, d_tbl%alts
  if (d_tbl%alt(ic,ialt) /= "-" .and. d_tbl%act_hit(ialt) == "y") then
    if (d_tbl%alt(ic,ialt) == "<"  .and. var_cur >= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == ">"  .and. var_cur <= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "<=" .and. var_cur >  var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == ">=" .and. var_cur <  var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "="  .and. var_cur /= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "/=" .and. var_cur == var_tbl) d_tbl%act_hit(ialt) = "n"
  end if
end do
```

After every condition has run, an alternative still marked `y` is a matched alternative. Any number of alternatives can match.

Action firing

The driver `actions.f90` walks `d_tbl%act(1:acts)`. For each action it scans alternatives:

```
do iac = 1, d_tbl%acts
  action = "n"
  do ial = 1, d_tbl%alts
    if (d_tbl%act_hit(ial) == "y" .and. d_tbl%act_outcomes(iac,ial) == "y") then
      action = "y"
      exit
    end if
  end do
  if (action == "y") then
    select case (d_tbl%act(iac)%typ)
      case ("manure_demand") ...
      case ("irr_demand")    ...
      case ("irrigate")      ...
      case ("fertilize")     ...
      case ("manure")        ...
      case ("till")          ...
      case ("plant")         ...
      case ("harvest")       ...
      case ("harvest_kill")  ...
      case ("kill")          ...
      case ("burn")          ...
      case ("graze")         ...
      case ("pest_apply")    ...
      case ("lu_change")     ...
      case ("weir_release")  ...
```

```
      case ("res_release")   ...
      ! many more
    end select
  end do
end do
```

The fire decision is "any matched alternative wants this action": as soon as one surviving alternative has `y` in the action-outcome cell, the action fires. The action-outcome matrix lets you write rules where some alternatives share actions and others differ.

For management actions, the action implementation calls the relevant operation routine. Examples:

- `case ("plant")` resolves the plant from the community database, sets the start of the heat-unit accumulation, and calls the planting initialiser.
- `case ("fertilize")` looks up `d_tbl%act_typ(iac)` in `fertdb`, applies the fertiliser, and updates pools through `pl_fert`.
- `case ("till")` looks up the tillage implement, calls `mgt_newtillmix*` to redistribute residue and nutrients.
- `case ("harvest")` looks up the harvest operation type and dispatches to `mgt_harvbiomass`, `mgt_harvgrain`, `mgt_harvresidue`, `mgt_harvtuber`, or related routines.
- `case ("lu_change")` calls `hru_lum_init` and `chg_par` to switch the land use class on the HRU and re-initialise dependent state.

For reservoir release actions, the action implementation lives in `res_hydro.f90` rather than `actions.f90`. See Reservoirs and wetlands.

Probability and land use change

Two condition variables modify the alternative narrowing with random draws:

- `prob` and `prob_unif` compare a uniform random number to a constant probability. The probability passes through `d_tbl%frac_app`.
- `land_use` paired with a probabilistic action distributes applications across HRUs of a land-use class. The reader counts HRUs in the matching class (`hru_lu`, `ha_lu`) and the runtime tracks how many have been applied (`hru_lu_cur`, `hru_act_left`).

This is how stochastic management (apply fertiliser to 30 percent of corn HRUs in May, for instance) is implemented.

Switches and parameters

Decision tables are configured through the `*.dtl` input files, not through `codes.bsn` or `parameters.bsn` switches. The behaviour of each table is controlled by:

Parameter	File	Effect
`dtbl%name`	`*.dtl`	Table name referenced by HRUs, reservoirs, or scenarios
`dtbl%conds`	`*.dtl`	Number of conditions
`dtbl%alts`	`*.dtl`	Number of alternatives (rule columns)
`dtbl%acts`	`*.dtl`	Number of actions
`dtbl%cond%var`	`*.dtl`	Condition variable name; see case list in `conditions.f90`
`dtbl%cond%ob`	`*.dtl`	Object the condition is evaluated against
`dtbl%cond%lim_const`	`*.dtl`	Constant value the condition variable is compared to
`dtbl%cond%lim_var`	`*.dtl`	Limit-variable name (e.g. `pvol`, `evol`, `fc`, `ul`)
`dtbl%cond%lim_op`	`*.dtl`	Limit operator (`*`, `+`, `-`)
`dtbl%alt(ic, ialt)`	`*.dtl`	Comparison operator for each condition under each alternative
`dtbl%act%typ`	`*.dtl`	Action type; see case list in `actions.f90`
`dtbl%act%ob`	`*.dtl`	Object the action is applied to
`dtbl%act%name`	`*.dtl`	Action name (e.g. operation name in `harv.ops`, `fert.frt`, …)

Parameter	File	Effect
`dtbl%act_outcomes(iac, ialt)`	`*.dtl`	`y` or `n` whether action `iac` fires under alternative `ialt`

Implementation

Source modules in `swatplus/src/`:

- `conditional_module.f90` defines `conditions_var`, `actions_var`, and `decision_table` derived types, plus the four runtime arrays `dtbl_lum`, `dtbl_res`, `dtbl_scen`, `dtbl_flo` and the working pointer `d_tbl`.
- `dtbl_lum_read.f90` reads `lum.dtl` into `dtbl_lum`. Allocates conditions, alternatives, actions, and outcome matrices, resolves named operation pointers (`act_typ`) to database indices.
- `dtbl_res_read.f90` reads `res_rel.dtl` into `dtbl_res`.
- `dtbl_scen_read.f90` reads `scen_lu.dtl` into `dtbl_scen`.
- `dtbl_flocon_read.f90` reads `flo_con.dtl` into `dtbl_flo`.
- `conditions.f90` evaluates conditions for the current `d_tbl`. Dispatches on `d_tbl%cond(ic)%var` and calls `cond_real` or `cond_integer` for narrowing.
- `cond_real.f90`, `cond_real_c.f90`, `cond_integer.f90`, `cond_integer_c.f90` implement the comparison operators and update `d_tbl%act_hit`.
- `actions.f90` fires actions whose `act_outcomes` entry is `y` for any surviving alternative. Houses the management action select-case (plant, fert, till, harvest, kill, burn, graze, lu_change, …).
- `hru_dtbl_actions_init.f90` initialises decision table linkages on HRUs at simulation start.
- `res_rel_conds.f90` is the reservoir-specific conditions evaluator used inside `res_hydro.f90`.

The call site sets `d_tbl => dtbl_lum(idtl)` (or `dtbl_res`, `dtbl_scen`, `dtbl_flo`) so that `conditions.f90` and `actions.f90` operate on the right table.

Related pages

- `*.dtl` input file for the file format used by all four classes.
- Reservoirs and wetlands for release rules driven by `res_rel.dtl`.
- Plant growth and land use management for the management actions fired by `lum.dtl`.